

Chapter 1

Introduction

1.1 Introduction

Virtualization and containerization technologies, combined with microservices architecture, have fundamentally transformed modern software development and infrastructure management. Virtualization enables multiple virtual machines (VMs) to run on a single physical server, significantly improving hardware utilization from typical rates of 15–20% to 70–80% or higher, thereby reducing hardware costs, power consumption, and data center footprint while enabling backup, cloning, and migration capabilities essential for disaster recovery.

Containerization extends these benefits through lightweight isolation mechanisms. Unlike virtual machines that require full operating systems, containers share the host kernel while maintaining isolated user spaces, allowing hundreds of containers to run on a single host. Docker has revolutionized application packaging by bundling code with dependencies into portable artifacts that run consistently across all environments, eliminating configuration drift and deployment failures. Kubernetes provides enterprise-grade orchestration with automated deployment, scaling, load balancing, and self-healing capabilities.

Microservices architecture decomposes monolithic applications into small, independent services communicating via application programming interfaces (APIs). Each microservice handles specific business capabilities and can be developed, deployed, and scaled independently. This architectural style enables parallel team development without coordination overhead, technology heterogeneity where teams choose optimal tools per service, and multiple daily releases without system-wide deployments. Fault isolation prevents cascading failures, allowing graceful degradation rather than complete outages, while selective scaling optimizes resources by scaling only high-load services.

Operationally, modern container platforms provide automated horizontal scaling based on metrics, self-healing through automatic container restarts and traffic routing, zero-

downtime rolling updates with automatic rollback, and declarative Infrastructure as Code (IaC) approaches. Advanced deployment strategies such as blue-green deployments, canary releases, and A/B testing reduce risk and enable data-driven decisions. For business strategy, containerized microservices support hybrid and multi-cloud deployments, providing consistent runtime environments across on-premises and cloud providers, reducing vendor lock-in and enabling workload optimization.

These patterns align with DevOps practices, allowing autonomous teams to own services end-to-end, reducing handoffs and communication overhead. Continuous integration and deployment (CI/CD) pipelines enable rapid iteration essential for competitive advantage. Modern monitoring and distributed tracing tools provide observability for complex distributed systems. While distributed systems introduce complexity, mature tooling including managed Kubernetes services, service meshes, and observability platforms abstract operational challenges. For organizations building or modernizing applications, these technologies provide a proven foundation for scalable, resilient, maintainable software that evolves with business requirements while optimizing infrastructure costs and development velocity.

1.2 Advanced Infrastructure

1.2.1 Description of Virtualized and Containerized Environments

Virtualization Technology Fundamentals

Virtualization represents a foundational paradigm shift in computing infrastructure that abstracts physical hardware resources into logical units, enabling multiple isolated computing environments to coexist on a single physical machine. The hypervisor serves as the critical component, sitting between physical hardware and virtual machines to manage resource allocation and maintain isolation. Two primary types exist: Type 1 hypervisors such as VMware ESXi, Microsoft Hyper-V, and Xen run directly on hardware without a host operating system, providing superior performance for production environments. Type 2 hypervisors including VMware Workstation and VirtualBox run as applications on host operating systems, offering ease of use for development and testing scenarios.

Virtual machines represent complete computing environments with virtualized processors, memory, storage, and network interfaces. Each VM runs its own operating system, completely isolated from others on the same host. The hypervisor manages CPU scheduling and memory allocation, employing sophisticated techniques such as memory ballooning to dynamically reclaim unused memory and page sharing to consolidate identical memory pages across VMs. Storage virtualization abstracts physical storage into logical volumes with features including thin provisioning that allocates storage on demand, snapshots for point-in-time recovery, and live migration capabilities enabling movement of running VMs

between physical hosts. Network virtualization creates virtual adapters, switches, and routers implemented entirely in software, enabling complex network topologies without requiring physical network equipment.

Resource management in virtualized environments involves sophisticated scheduling algorithms that optimize hardware utilization. CPU oversubscription allocates more virtual CPUs than physical cores exist, relying on the statistical likelihood that not all VMs will require maximum CPU simultaneously. Memory overcommitment similarly allocates more memory than physically available, utilizing compression and swapping techniques during periods of high demand. Resource pools enable collective management of computational resources, with shares determining relative priority, reservations guaranteeing minimum resources, and limits constraining maximum consumption. These mechanisms allow administrators to implement quality-of-service policies ensuring critical applications receive required resources even under contention.

High availability features distinguish enterprise virtualization platforms from basic hypervisors. VM live migration enables moving running virtual machines between physical hosts without downtime, facilitating hardware maintenance and load balancing. Fault tolerance maintains synchronized copies of VMs on separate hosts, enabling automatic failover within seconds if primary hosts fail. Distributed Resource Scheduler (DRS) continuously monitors cluster resource utilization, automatically migrating VMs to balance workloads and prevent resource exhaustion on individual hosts. These capabilities transform commodity hardware into highly available infrastructure previously requiring specialized equipment.

Containerization Architecture and Implementation

Containerization provides an alternative isolation approach that virtualizes at the operating system level rather than the hardware level, enabling fundamentally different performance characteristics and resource efficiency. Containers share the host kernel while maintaining isolated user spaces through kernel features, achieving densities and startup times impossible with traditional virtualization. A single host supporting perhaps a dozen VMs can run hundreds of containers, as containers avoid the overhead of complete operating systems and hypervisor layers.

Linux containers rely on two primary kernel features for isolation and resource management. Namespaces isolate various system aspects: process ID namespaces ensure containers cannot view or signal processes in other containers, network namespaces provide separate network stacks with independent routing tables and firewall rules, mount namespaces isolate filesystem views, user namespaces map container root users to non-privileged host users for security, inter-process communication namespaces isolate message queues and shared memory, and Unix Time-Sharing (UTS) namespaces allow different hostnames per container. Control groups (cgroups) provide resource limitation and accounting, con-

App A	App B	App C
Bins/Libs	Bins/Libs	Bins/Libs
Container Engine (e.g., Docker)		
Host OS		
Hardware Infrastructure		

App A	App B	App C
Bins/Libs	Bins/Libs	Bins/Libs
Guest OS	Guest OS	Guest OS
Hypervisor		
Host OS		
Hardware Infrastructure		

Figure 1.1: Architectural comparison between Hardware Virtualization (Virtual Machines) and Operating System Virtualization (Containers).

straining CPU usage, memory allocation, block I/O bandwidth, and network bandwidth per container, preventing any single container from monopolizing host resources.

Union filesystems enable efficient container storage through layered filesystem implementations. Base images containing operating system files and libraries are shared read-only across all containers using that image, dramatically reducing storage requirements and improving cache efficiency. Each container adds a thin writable layer on top of the shared base, with copy-on-write mechanisms ensuring modifications remain isolated while avoiding duplication until changes occur. This layering approach means launching a new container requires only creating a new writable layer, enabling sub-second startup times regardless of image size.

Docker revolutionized containerization by providing a complete platform ecosystem for building, distributing, and running containers. Dockerfiles specify image construction through declarative instructions, creating reproducible builds that eliminate environment-specific configuration issues. The Docker daemon manages container lifecycle operations including creation, starting, stopping, and removal, while exposing APIs for integration with orchestration platforms. Container registries like Docker Hub provide centralized image repositories for distribution, enabling developers to share pre-built images and organizations to maintain private registries for proprietary applications.

Container networking addresses fundamental challenges arising from ephemeral container lifecycles through multiple networking modes. Bridge networking creates virtual network segments with Network Address Translation (NAT) for external connectivity, isolating containers while enabling internet access. Host networking bypasses isolation to share the host's network stack directly, providing maximum performance for network-intensive applications at the cost of reduced isolation. Overlay networks span multiple physical hosts, enabling containers on different machines to communicate as if on the same network segment, essential for distributed applications. Volume management solves data persistence challenges through Docker volumes providing managed storage external to container filesystems, bind mounts enabling development workflows requiring host filesystem access, and volume plugins integrating enterprise storage systems.

Container Orchestration with Kubernetes

Kubernetes emerged as the industry standard for container orchestration, providing declarative infrastructure management where administrators specify desired state and the system continuously reconciles actual state toward those specifications. This approach represents a fundamental shift from imperative management where operators manually execute commands to achieve desired configurations. The architecture consists of a control plane managing cluster state and worker nodes running containerized workloads.

The control plane components coordinate all cluster operations. The API server provides the central management interface, exposing RESTful APIs for all cluster interactions

and serving as the only component directly accessing the cluster state store. Etcd stores all cluster state in a distributed, strongly consistent key-value store, providing reliable persistence and watch capabilities for change notifications. The scheduler assigns newly created pods to worker nodes based on resource requirements, constraints, affinity specifications, and current cluster utilization. Controller managers run control loops that continuously observe cluster state and make changes to move current state toward desired state, implementing core functionality like replica management, endpoint updates, and service account creation.

Worker nodes execute the actual application workloads. The kubelet agent runs on each node, managing pod lifecycle by ensuring containers described in pod specifications are running and healthy. The container runtime executes containers, with Kubernetes supporting multiple runtimes including Docker, containerd, and CRI-O through a standard interface. Kube-proxy manages network routing, implementing service abstractions by maintaining network rules that enable communication with pods regardless of their current node placement.

Pods represent the smallest deployable units in Kubernetes, encapsulating one or more containers sharing network and storage namespaces. Containers within pods communicate via localhost, share storage volumes, and are scheduled together on the same node. Deployments manage stateless application rollout, providing declarative updates, automated rollbacks on failure, and scaling capabilities. StatefulSets handle stateful applications requiring stable network identities, ordered deployment and scaling, and persistent storage that follows pods across rescheduling. Services provide stable networking endpoints abstracting dynamic pod IP addresses, with built-in load balancing distributing traffic across healthy pods matching label selectors.

Configuration management in Kubernetes separates application code from environment-specific configuration. ConfigMaps store non-sensitive configuration data as key-value pairs or files, enabling the same container images to run across environments with different configurations. Secrets provide similar functionality for sensitive information such as passwords, OAuth tokens, and SSH keys, with additional features including encryption at rest and access controls. Applications consume configurations through environment variables, command-line arguments, or mounted configuration files.

Resource management ensures efficient cluster utilization while preventing resource contention. Resource requests specify minimum guaranteed resources for each container, influencing scheduling decisions to place pods on nodes with sufficient capacity. Resource limits define maximum consumable resources, with enforcement preventing containers from exceeding allocations and impacting other workloads. Quality-of-service classes derive from request and limit configurations, determining eviction priority when nodes experience resource pressure. Horizontal Pod Autoscalers automatically scale deployments based on observed CPU utilization or custom metrics, adding replicas during high load and

removing them during low utilization. Namespaces provide logical cluster partitioning, enabling multi-tenancy by isolating resources and allowing independent resource quotas and access controls per namespace. Network policies control pod communication, implementing microsegmentation by specifying which pods can communicate with each other and which external endpoints are accessible.

1.2.2 Advanced Development of Microservices and Architectural Comparison

Microservices Architecture Principles

Microservices architecture structures applications as collections of loosely coupled services, each implementing specific business capabilities independently deployable and scalable. This architectural style represents a fundamental departure from monolithic design, distributing functionality across multiple autonomous services communicating through well-defined interfaces. Services maintain independent data stores following the database-per-service pattern, ensuring loose coupling at the data layer and enabling technology diversity where different services can utilize databases optimized for their specific requirements.

Each service team owns the complete service lifecycle including development, deployment, monitoring, and maintenance, fostering ownership and accountability. The bounded context principle from domain-driven design guides service boundary identification, ensuring services encapsulate cohesive business capabilities with clear responsibility boundaries and minimal external dependencies. Properly identified boundaries minimize inter-service communication while maximizing team autonomy, critical factors for microservices success.

Service independence enables independent deployment without coordinating across teams or services, dramatically accelerating release velocity compared to monolithic deployments requiring system-wide coordination. Teams can choose optimal technology stacks for their specific requirements rather than conforming to organization-wide standards. This heterogeneity allows polyglot programming where services use different programming languages, databases, and frameworks based on specific functional and non-functional requirements. Payment processing might utilize strongly-typed languages for correctness guarantees, while recommendation engines might employ machine learning frameworks requiring different runtime environments.

Team autonomy reduces cognitive load, as developers can understand and modify individual services without comprehending entire system architectures. New team members achieve productivity rapidly by focusing on specific service domains rather than learning vast interconnected codebases. This localized understanding enables faster onboarding and reduces coordination overhead across organizational boundaries.

Resilience patterns handle inevitable failures in distributed systems through deliberate design. Circuit breakers prevent cascading failures by detecting failing downstream services and short-circuiting requests, giving failing services time to recover without overwhelming them with additional traffic. Bulkheads isolate resources so failures in one service or component do not consume resources needed elsewhere, preventing localized failures from causing system-wide outages. Retry logic with exponential backoff handles transient failures gracefully by automatically retrying failed operations with increasing delays between attempts. Timeouts prevent indefinite waiting on unresponsive services, ensuring requests fail quickly rather than exhausting resources. These patterns combine to create antifragile systems that improve under stress rather than degrading catastrophically.

Data Management in Distributed Systems

Data consistency in microservices requires abandoning traditional ACID transactions spanning multiple services, as distributed transactions introduce coordination overhead incompatible with service independence. The saga pattern coordinates distributed transactions through sequences of local transactions with compensating actions for rollback. Choreography-based sagas use event publishing for loose coupling, while orchestration-based sagas employ central coordinators for better visibility.

Event sourcing stores state changes as immutable event sequences, providing audit trails and temporal queries. Command Query Responsibility Segregation (CQRS) separates read and write models for independent optimization. Event-driven architectures enable eventual consistency where updates propagate asynchronously, accepting temporary inconsistency for availability per the CAP theorem. Data replication balances consistency and availability: strong consistency ensures identical replicas but reduces availability during partitions, while eventual consistency allows divergence for high availability.

Communication Patterns and Service Integration

Microservices require robust communication mechanisms for service interaction, with architectural choices profoundly impacting system characteristics. Synchronous Representational State Transfer (REST) APIs provide request-response communication with clear semantics, widespread tooling support, and simple mental models. Services expose HTTP endpoints receiving requests and returning responses, enabling straightforward integration and debugging. However, synchronous communication creates temporal coupling where requesting services must wait for responses, reducing availability when downstream services fail or experience latency. Cascading failures occur when slow services create backpressure throughout the call chain, potentially bringing down entire service graphs.

Asynchronous messaging decouples services temporally, allowing producers to publish messages without waiting for consumer processing. Message brokers such as RabbitMQ,

Apache Kafka, or cloud-native services provide reliable message delivery, persistence during consumer downtime, and sophisticated routing capabilities. Event-driven architectures publish domain events that interested services consume, creating loose coupling where publishers remain unaware of their consumers. This approach enables flexible system evolution where new services can begin consuming existing events without modifying producers, and existing consumers can be updated independently without affecting publishers.

Service discovery mechanisms allow services to locate each other dynamically in containerized environments where instances constantly start, stop, and migrate across hosts. Client-side discovery has services query discovery registries like Consul or Eureka to obtain current service locations before making requests. Server-side discovery uses load balancers or API gateways that query registries and route requests, abstracting discovery complexity from clients. Service mesh implementations such as Istio or Linkerd provide sophisticated service-to-service communication through sidecar proxies deployed alongside each service instance, handling discovery, load balancing, circuit breaking, mutual TLS encryption, and observability transparently without requiring application code changes.

API gateways consolidate external access, providing a single entry point for client applications. Gateways handle cross-cutting concerns including authentication, authorization, rate limiting, request routing, response aggregation, and protocol translation. They can aggregate multiple backend service calls into single responses, reducing client complexity and network round trips. Backend for Frontend (BFF) patterns create specialized gateways for different client types such as web applications, mobile applications, and partner APIs, optimizing responses and APIs for specific client requirements and capabilities.

Comparative Analysis: Monolithic vs Microservices Architecture

The fundamental architectural choice between monolithic and microservices approaches profoundly impacts development practices, operational complexity, and business outcomes. Monolithic architectures package all functionality into a single deployable unit sharing one codebase, database, and runtime. This offers simplicity: development uses familiar integrated codebases with standard debugging tools, transaction management relies on database ACID properties ensuring consistency, and deployment involves single artifacts using established processes.

Monolithic architectures excel for small teams with limited scale requirements. Shared codebases facilitate code reuse and refactoring. Single databases simplify queries and ensure consistency. Simple deployment reduces operational burden compared to coordinating multiple services. For startups and small teams, monolithic architectures often represent the optimal choice, avoiding complexity that would slow development without commensurate benefits.

However, monolithic architectures encounter limitations as systems grow. Scaling requires replicating entire applications regardless of component load, leading to inefficient resource utilization. Technology choices become embedded, preventing adoption of new tools without complete rewrites. Deployment risk increases with application size, as any change necessitates full redeployment, creating extended testing cycles. Large codebases become difficult to understand as complexity accumulates, reducing development velocity.

Microservices decompose applications into independent services with isolated data stores. This provides scaling flexibility for specific high-load services while maintaining minimal footprints elsewhere, optimizing costs and performance. Technology diversity enables incremental adoption without system-wide migrations. Independent deployment accelerates release velocity, enabling daily deployments without organizational coordination. Fault isolation contains failures, enabling graceful degradation.

However, microservices introduce significant complexity. Distributed challenges include network latency affecting interactions, partial failures requiring deliberate handling, and data consistency across boundaries through complex patterns. Operational overhead increases with monitoring, logging, and tracing requirements. Deployment complexity grows with version management, schema migrations, and backward compatibility. Development complexity includes designing service boundaries, implementing distributed transactions, and reasoning about eventual consistency.

The decision framework considers multiple factors. Team size influences selection: small teams often lack resources for distributed complexity, while large organizations benefit from service autonomy. System scale drives choices: applications with uniform load may not benefit from selective scaling, while those with varying requirements justify microservices complexity. Domain complexity affects decisions: clearly separable capabilities align well with microservices boundaries, while tightly coupled domains suffer from excessive communication. Organizational maturity determines success: microservices demand sophisticated DevOps practices and operational expertise.

1.3 Problematic

Modern software systems face unprecedented demands for scalability, reliability, and rapid deployment in competitive digital markets. Organizations must navigate critical architectural decisions that fundamentally shape technology strategies, development practices, and business outcomes. The choice between traditional monolithic architectures and containerized microservices environments represents a strategic transformation affecting organizational structure, operational processes, infrastructure investments, and competitive positioning.

Traditional monolithic architectures offer development simplicity, straightforward debugging, and simple deployment through single artifacts. However, they encounter

fundamental limitations as systems grow: scaling requires replicating entire applications regardless of specific component load, technology choices become locked in, deployment risk increases with system size, and large codebases impede development velocity. Coordinating changes across large teams generates merge conflicts and communication overhead, while technical debt accumulates as shortcuts create ripple effects through tightly coupled systems.

Containerized microservices promise solutions through service independence enabling autonomous team operation, selective scaling optimizing resource utilization, technology heterogeneity allowing incremental adoption of new tools, and fault isolation preventing cascading failures. Yet these benefits come with substantial complexity: distributed systems introduce network latency and partial failures, operational overhead increases with monitoring requirements across numerous services, deployment complexity grows with version coordination, and organizational transformation requires team restructuring and skill development.

Problem Statement:

What technical, operational, and business factors obligate organizations to transition from traditional monolithic architectures to virtualized and containerized environments with microservices? Under what circumstances do scalability limitations, deployment constraints, and organizational growth necessitate this architectural shift despite substantial increases in system complexity and operational overhead? How do organizations evaluate trade-offs between monolithic simplicity and microservices flexibility to determine appropriate architecture for specific contexts, and what migration strategies minimize transformation risk while delivering incremental business value?

Chapter 2

Conception

2.1 Introduction

Designing a cluster infrastructure capable of hosting a high-load web application while simultaneously guaranteeing service availability and controlling operational costs requires decisions that span multiple abstraction levels, from the physical placement of servers and network equipment down to the scheduling of individual containers. This chapter presents the conception of such an infrastructure across three tightly coupled pillars. The first is the *proof of concept*, which establishes the functional objectives and validation conditions for the proposed architecture before full-scale deployment is committed. The second is *monitoring and service placement*, which defines the layered architecture of the cluster — physical, virtual, containerisation, and monitoring — together with the scheduling, scaling, and placement algorithms that govern runtime resource allocation. The third is *artificial intelligence and security*, which introduces the intelligent assistance and predictive mechanisms that augment operator capabilities and the security posture that protects the entire stack.

2.2 Proof of Concept

2.2.1 Context and Objectives

The target workload is a high-load web application whose demand characteristics require both elastic scalability and continuous availability. Two operational requirements define the design space. The first is *cost efficiency*: physical resources are expensive, and their utilisation must be maximised by placing workloads densely and scaling them precisely in response to actual demand rather than provisioning conservatively for peak load at all times. The second is *high availability*: users must not experience service interruption due to hardware failures, software faults, or traffic surges. These two requirements are in structural tension — dense placement increases the blast radius of individual node failures,

while conservative redundancy wastes resources — and the architecture must resolve that tension through intelligent scheduling rather than by sacrificing one objective for the other [?, ?].

The application is structured as a collection of loosely coupled microservices, each responsible for a distinct functional capability and deployable independently of the others. This decomposition is a precondition for the scaling and placement strategies developed in the following sections: a monolithic application can only be scaled as a unit, whereas individual microservices can be replicated or resized in response to the specific resource pressures they experience. Microservice independence also ensures that a fault in one functional component causes graceful degradation rather than total service failure, directly supporting the high-availability objective [?].

2.2.2 Validation Scope

The proof of concept is a bounded experimental phase whose purpose is to verify that the core architectural mechanisms — VM provisioning, container scheduling, inter-service communication, and scaling reactions — function correctly together before the full cluster is assembled. The PoC is considered successful when four conditions are met simultaneously. First, the provisioning pipeline must onboard a new virtual machine into the cluster without manual intervention and within a defined time bound. Second, the scaling mechanisms must react to injected load anomalies — elevated error rates, CPU saturation, memory pressure, or I/O exhaustion — by applying the correct remediation within the latency budget of the monitoring cycle. Third, the container scheduling layer must distribute workloads across the virtual layer without violating resource constraints and must rebalance automatically when a virtual machine is added or removed. Fourth, the application must remain reachable throughout all provisioning and scaling operations, confirming that the high-availability objective is satisfied under the stress conditions most likely to occur in production.

2.3 Monitoring and Service Placement

The cluster is structured as four superimposed layers, each providing services consumed by the layer above it. The physical layer provides raw compute, storage, and network capacity. The virtual layer abstracts that capacity into manageable virtual machines and organises them into logical groupings. The containerisation layer deploys application workloads as containers distributed across the virtual machines. The monitoring layer observes the state of all three operational layers and drives the automated responses that maintain the desired operational condition of the cluster.

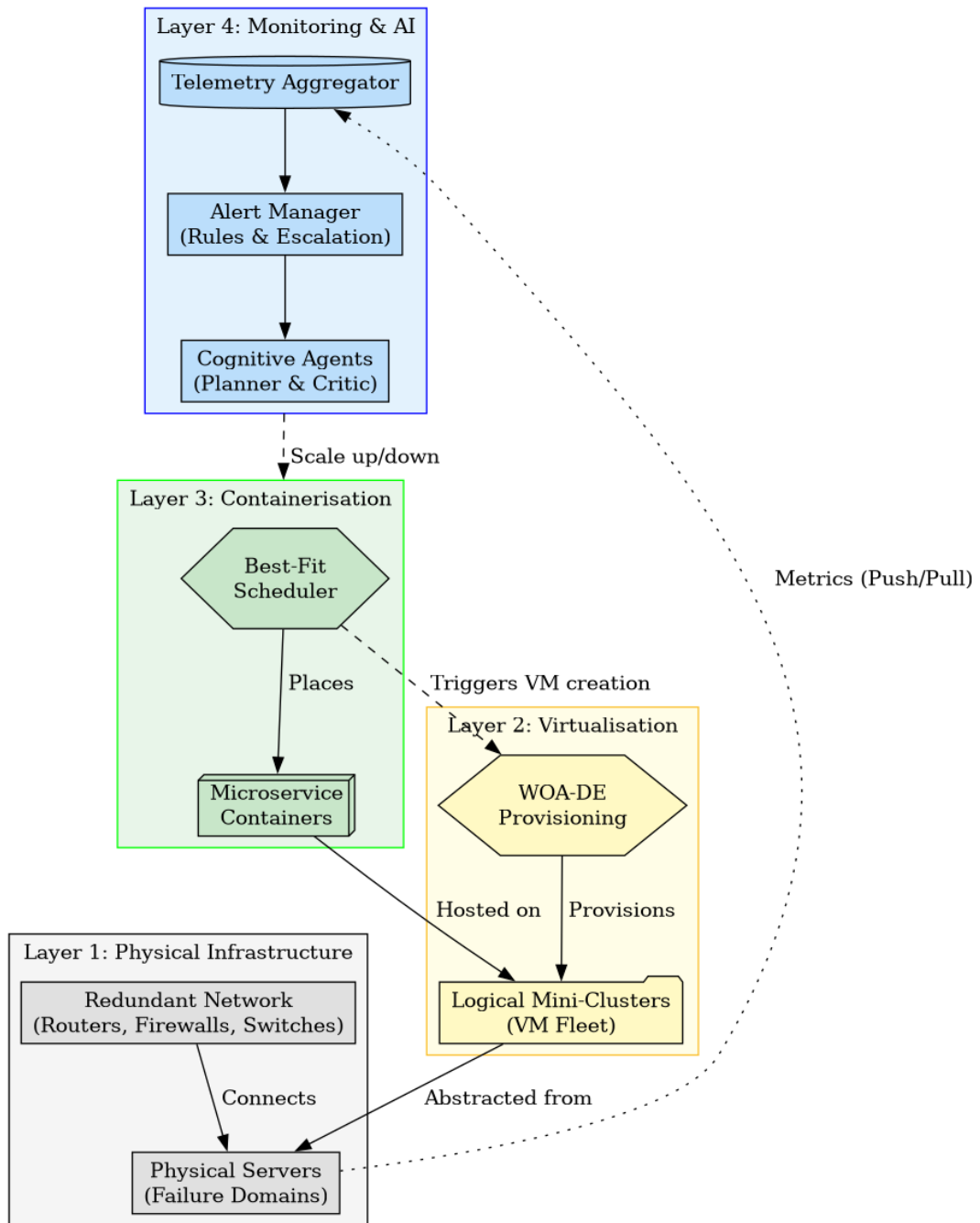


Figure 2.1: The hierarchical 4-layer architecture of the full cluster ecosystem.

2.3.1 Physical Layer

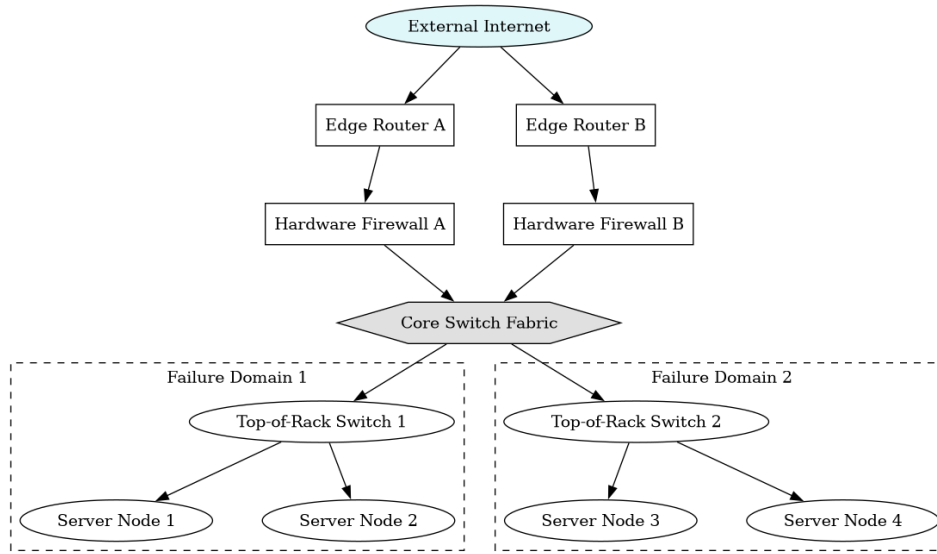


Figure 2.2: Layer 1: Physical Infrastructure, emphasizing explicit failure domains and redundant networking.

The physical layer is the hardware substrate of the cluster. It consists of a set of physical servers that provide the compute and storage capacity on which all virtual and containerised workloads ultimately execute. Servers are connected through a switching fabric composed of managed network switches that enforce internal traffic segmentation and carry inter-VM and inter-container communication with bounded latency. At the boundary between the internal cluster network and the external internet, a set of hardware firewalls enforces ingress and egress traffic policies, filtering malformed or unauthorised requests before they reach internal services. Routers handle inter-segment routing within the cluster and manage the uplink to external networks.

The physical layer is designed around explicit *failure domains*: groups of servers that share a common risk of simultaneous failure, such as nodes connected to the same power supply or the same top-of-rack switch. Workload replicas are distributed across distinct failure domains so that no single hardware event can eliminate all instances of a service. Network equipment is deployed in redundant pairs so that the failure of any single switch, router, or firewall does not partition the cluster or interrupt traffic to hosted services.

2.3.2 Virtual Layer

The virtual layer abstracts the physical server resources into a fleet of virtual machines, each representing an isolated execution environment with a defined allocation of virtual CPUs, memory, disk, and network bandwidth. Virtual machines are provisioned, sized, and placed on physical servers by the mechanisms described in the following subsections.

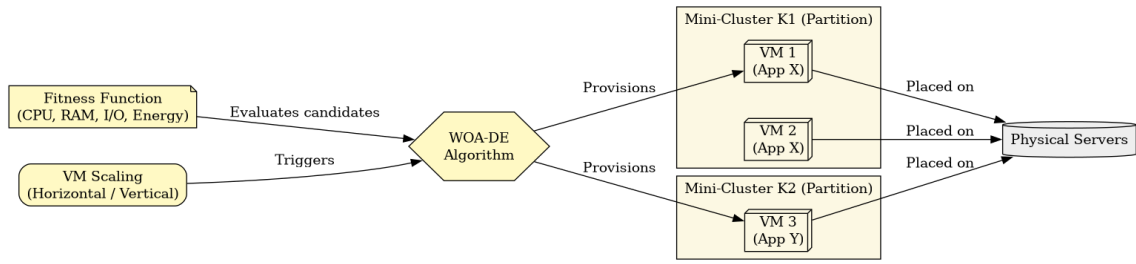


Figure 2.3: Layer 2: Virtualization, highlighting the WOA-DE Provisioning Algorithm mapping VMs to Physical Servers.

VM Sizing

The initial resource envelope of a newly created virtual machine is derived from the workload profile of the service it will host. A workload profile characterises the expected resource demand of a service based on performance benchmarks, declared service-level objectives, or historical utilisation data from similar services already running in the cluster. The baseline is not fixed: the monitoring layer continuously tracks per-VM resource consumption and the VM may be vertically scaled at runtime if observed utilisation consistently exceeds defined thresholds, as described in the scaling subsection below. A set of standard size classes is defined on top of this baseline, providing pre-validated resource ratios for the most common workload types and simplifying provisioning decisions for well-characterised services.

Mini-Cluster Partitioning

A key structural contribution of this architecture is the partitioning of the VM fleet into k *logical mini-clusters*. The motivation for this decomposition is both algorithmic and operational. Algorithmically, applying the VM provisioning algorithm over the full set of n physical machines has a complexity proportional to n ; by restricting each scheduling decision to the n/k physical machines that belong to a candidate mini-cluster, the effective scheduling complexity is reduced by a factor of k , a significant gain at large fleet sizes [?]. Operationally, mini-clusters group virtual machines whose container workloads are strongly related — that is, microservices that communicate frequently or share data — so that the majority of inter-service traffic is confined within a single mini-cluster, minimising cross-segment network overhead and simplifying traffic policy management.

The partitioning further ensures an equitable distribution of containers and of the physical resources allocated to them across mini-clusters, preventing hotspots from forming in any single segment of the fleet. A long-term benefit of this organisation is that virtual machines within the same mini-cluster exhibit similar resource consumption patterns, making collective resource adjustment decisions more tractable as the fleet grows and as application complexity increases over time.

The number of mini-clusters k is determined by one of two strategies. In the **static** approach, k is derived from the Key Performance Indicators (KPIs) defined by the application designer prior to deployment, reflecting a known understanding of the service dependency graph. In the **dynamic** approach, k is inferred and updated continuously by a machine learning model that observes inter-service communication patterns and resource consumption correlations, adjusting the partitioning as the application evolves [?, ?]. The dynamic approach is the long-term target; the static approach provides a reliable starting point when historical data is unavailable.

VM Provisioning Algorithm

The VM provisioning algorithm addresses the following problem: given a virtual machine with a defined resource request, select the most appropriate physical machine within the relevant mini-cluster on which to place it. The algorithm is a purpose-built hybrid that draws conceptual inspiration from two established meta-heuristic optimisation strategies — the **Whale Optimisation Algorithm** (WOA) [5] and **Differential Evolution** (DE) [6]. The adaptation of such evolutionary models has proven highly effective for optimal VM allocation in cloud environments [9]. Rather than transplanting both algorithms wholesale, the design selects the mechanisms from each that best suit the structural constraints of the VM placement problem, combining them into a single coherent procedure. Recent advances have demonstrated the efficacy of Whale Optimization in handling multi-objective VM placement securely [11].

Conceptual origins. The Whale Optimisation Algorithm [5] mimics the bubble-net hunting behaviour of humpback whales. A population of candidate solutions updates its positions by either contracting toward the current best solution (exploitation) or spiralling around it (exploration along a logarithmic spiral), with a stochastic switch between the two behaviours and a third mode — random search around an arbitrary population member — that prevents premature convergence. Differential Evolution [6] is a population-based optimiser that generates trial solutions through a mutation step (a weighted difference of two randomly chosen population members added to a third) followed by a crossover step that exchanges components between the trial and the original vector, retaining the survivor that achieves the better fitness score. Both algorithms maintain a population of candidate solutions and iteratively improve a globally tracked best solution, but they achieve diversity through fundamentally different mechanisms: WOA through position-update equations parametrised by distance to the best, DE through algebraic recombination of distinct population members.

Hybrid design rationale. The algorithm partitions the population at each iteration into two disjoint subsets. The first subset, of size n_{WOA} , is updated through WOA-style

position equations, benefiting from the algorithm's well-characterised balance between spiral exploitation and contraction-based convergence. The second subset, of size n_{DE} , is updated through DE-style mutation and crossover, using two population members not belonging to either subset as mutation vectors, and then competing the trial solution against its parent through selection. After both subsets have been updated, the globally tracked best solution X^* is refreshed to reflect whichever candidate in the entire population achieved the lowest fitness. This split prevents the entire population from converging prematurely toward X^* (a known weakness of pure WOA) while avoiding the directionless random walk that can arise in DE when the population diversity collapses.

Formal description. Let n denote the population size and T the number of iterations. Each candidate solution X_i represents a complete mapping of virtual machines to physical machines within the target mini-cluster, and its fitness is evaluated by the function described in the following subsection. The algorithm proceeds as follows.

Initialisation. Draw n candidate mappings $\{X_0, X_1, \dots, X_{n-1}\}$ uniformly at random from the space of feasible placements. Set X^* to the mapping with the lowest fitness among these n candidates. Set the WOA control parameter $a_0 = 2$ and the DE scaling factor F and crossover rate CR (with $CR = 0.5$ chosen for symmetry).

Main loop (for $t = 0, 1, \dots, T$).

1. **Update control parameters.**

$$a = 2\left(1 - \frac{t}{T}\right), \quad A = 2ar_1 - a, \quad C = 2r_2,$$

where $r_1, r_2 \sim \text{Uniform}(0, 1)$ are drawn independently at each iteration. A random probability $p \sim \text{Uniform}(0, 1)$ is also drawn.

2. **WOA update phase.** For each candidate X_i in the WOA subset:

- If $p < 0.5$:
 - If $|A| < 1$ (exploitation — shrinking encircling):

$$D = |C \cdot X^* - X_i(t)|, \quad X_i(t+1) = X^* - A \cdot D.$$

- If $|A| \geq 1$ (exploration — random agent): let X_{rand} be a randomly chosen member of the current population; then

$$D = |C \cdot X_{\text{rand}} - X_i(t)|, \quad X_i(t+1) = X_{\text{rand}} - A \cdot D.$$

- If $p \geq 0.5$ (spiral update):

$$D = |X^* - X_i(t)|, \quad X_i(t+1) = D e^{bl} \cos(2\pi l) + X^*(t),$$

where b is the logarithmic spiral constant and $l \sim \text{Uniform}(-1, 1)$.

After updating all members of the WOA subset, refresh X^* with the best solution found so far.

3. **DE mutation, crossover, and selection phase.** For each candidate X_j in the DE subset:

- (a) *Mutation.* Pick two distinct population members X_{r_1} and X_{r_2} that belong to neither the WOA subset nor the DE subset, and form the mutant vector

$$V_j = X^* + F \cdot (X_{r_1} - X_{r_2}).$$

Using X^* as the base vector biases the search toward the best-known region of the solution space.

- (b) *Crossover.* For each component k of the mapping:

$$U_j[k] = \begin{cases} V_j[k] & \text{if } \text{rand}() \leq CR, \\ X_j[k] & \text{otherwise.} \end{cases}$$

- (c) *Selection.* Evaluate the trial mapping U_j ; if $f(U_j) < f(X_j)$, replace X_j with U_j .

After processing all members of the DE subset, refresh X^* to the best solution among X^* itself and all updated X_j values.

Output. After $T + 1$ iterations, X^* is the recommended placement of all virtual machines onto physical machines within the mini-cluster.

Diagram.

Properties of the hybrid algorithm. The combination of WOA and DE phases produces a number of desirable characteristics for the VM placement setting.

- **Dual exploration mechanism.** The WOA spiral and random-agent modes explore the placement space from directions anchored to X^* and to random population members respectively, while the DE mutation explores directions defined by the algebraic difference $X_{r_1} - X_{r_2}$. These two mechanisms are geometrically independent: the first is driven by distance to the best, the second by intra-population diversity.

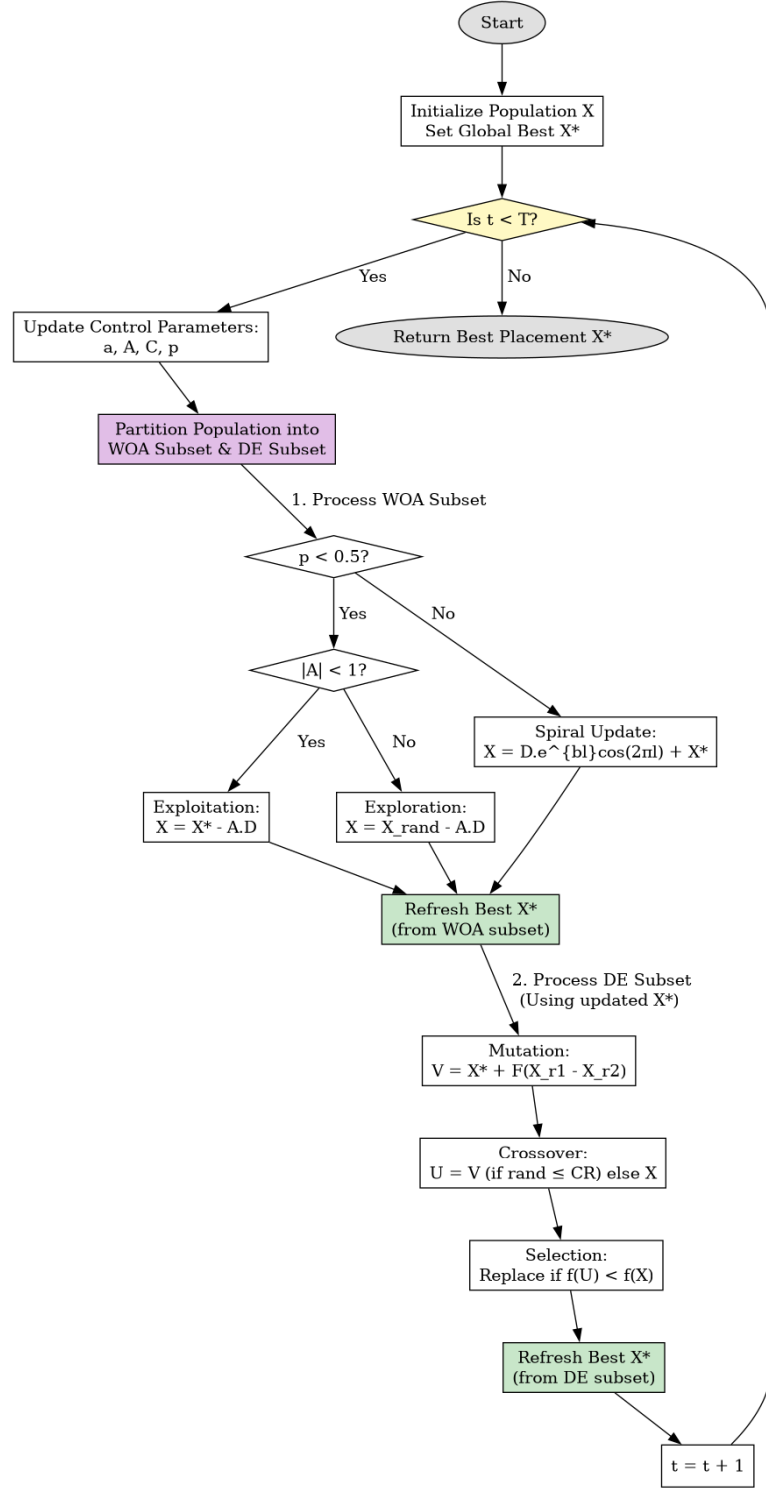


Figure 2.4: Flowchart of the hybrid WOA–DE provisioning algorithm, highlighting the parallel split and parameter integration loops.

Running them in parallel within the same iteration substantially reduces the risk that the search stagnates in a local optimum.

- **Preservation of the global best.** The WOA contraction mode ($|A| < 1$) and the DE base-vector choice (X^*) both orient the population toward the current best solution, ensuring that fitness improvements are not discarded between iterations. The refreshment of X^* after each phase further guarantees that the best candidate identified during the WOA phase can immediately influence the DE phase within the same iteration.
- **Greedy selection.** The DE selection step discards any trial mapping that does not improve upon its parent, so the population fitness is monotonically non-increasing across iterations. Combined with the WOA encircling contraction, this ensures that the algorithm converges to a locally optimal placement rather than oscillating indefinitely.
- **Parameter sensitivity reduction.** Pure WOA performance depends heavily on the spiral constant b and the scheduling of a , while pure DE is sensitive to the scaling factor F and the crossover rate CR . By mixing both paradigms, the hybrid distributes the optimisation workload across two parameter sets whose failure modes are uncorrelated, reducing the probability that a single poorly chosen parameter degrades the entire search.
- **Suitability for discrete placement spaces.** The position-update equations of both WOA and DE produce real-valued candidate solutions that must be rounded or decoded into feasible discrete mappings before evaluation. The fitness function (described below) naturally handles this discretisation, so the continuous update rules of both constituent algorithms apply without modification.

The principal limitation of the hybrid is its population-level computational cost: each iteration requires $O(n)$ fitness evaluations, which becomes significant when the number of physical machines in the cluster is large. A secondary limitation is that the position-update equations of both WOA and DE are defined over continuous spaces, so a decoding step is required to map real-valued candidate solutions back onto the discrete set of feasible VM-to-physical-machine assignments before each fitness evaluation.

Fitness Function

The fitness function quantifies how well a candidate physical machine satisfies the resource requirements of a virtual machine to be placed on it. Four resource dimensions are considered: CPU capacity (C), RAM (R), input/output throughput including secondary storage and network bandwidth (IO), and energy consumption (E). Because these metrics are

measured in heterogeneous units — CPU in cores, RAM in gigabytes, I/O in megabytes per second, and energy in joules per hour — direct comparison would assign disproportionate weight to high-magnitude dimensions. Each metric is therefore standardised prior to combination using the formula:

$$X_i^* = \frac{X_i - \bar{X}}{\sigma_X}$$

where $\bar{X}$ is the mean and σ_X is the standard deviation of metric X across all candidate physical machines in the sample. This normalisation ensures that each dimension contributes equally to the composite score regardless of its original unit or magnitude.

The composite fitness is then computed as a weighted Euclidean distance:

$$\text{fitness} = \sqrt{w_1 \cdot (\Delta C)^2 + w_2 \cdot (\Delta R)^2 + w_3 \cdot (\Delta IO)^2 + w_4 \cdot (\Delta E)^2}$$

where ΔX denotes the difference between the resource quantity requested by the virtual machine and the quantity currently available on the candidate physical machine. The squared terms give greater penalisation to large resource gaps than to small ones, encoding the operational priority of avoiding under-provisioned placements that would cause downtime over merely sub-optimal ones that waste some capacity. The weights w_i are defined by the application designer to reflect the relative importance of each resource dimension for the specific workload, subject to the normalisation constraint $\sum_{i=1}^4 w_i = 1$. A placement with lower fitness score is preferred, as it corresponds to a physical machine whose available resources more closely match the demands of the virtual machine being placed.

VM Scaling

The monitoring layer continuously evaluates per-VM resource utilisation and application-level error rates. When observed metrics cross defined thresholds, the alert manager triggers the appropriate scaling action. Two classes of scaling response are defined.

Horizontal scaling responds to conditions that indicate that the service is receiving more requests than a single VM instance can handle. It is triggered when the HTTP 5xx error rate exceeds 1% (warning) or 5% (critical), or when the network packet loss rate exceeds 2% (critical). The remediation creates a new virtual machine with the same resource configuration as the saturated instance; the new VM is placed within the same mini-cluster as its parent by invoking the provisioning algorithm, and it immediately joins the load-balancing pool for the corresponding service. Horizontal scaling preserves service availability by distributing demand across a larger number of equivalent instances rather than by interrupting the existing one.

Vertical scaling responds to resource saturation on an individual VM. CPU utilisation above 80% (warning) or 95% (critical) triggers the addition of one virtual CPU to the instance. RAM utilisation above 60% (warning) or 80% (critical) triggers an increase of 20% of the previously allocated memory. Free disk I/O capacity below 20% (both warning and critical thresholds) triggers the addition of 15 GB of storage. Vertical scaling modifies the resource envelope of the existing virtual machine without duplicating it, making it appropriate for bottlenecks that are intrinsic to the workload rather than caused by request volume.

In all scaling scenarios, if the physical machine hosting the virtual machine does not possess sufficient free capacity to accommodate the additional resources requested by vertical scaling, the virtual machine is live-migrated to a more suitable physical machine to mitigate performance interference and optimize energy consumption [8]. This migration is performed by invoking the provisioning algorithm on the migrating VM, which selects the best available physical machine within the mini-cluster according to the fitness function described above.

Virtual Network Architecture

Abstracting physical compute and storage is insufficient without a corresponding abstraction of the physical network. Without it, virtual machines placed on different physical servers remain mutually visible in ways that undermine isolation. The virtual network layer decouples logical topology entirely from physical cabling and switching, allowing segment boundaries to be defined and enforced in software on each hypervisor node, with no reconfiguration of physical equipment.

Overlay networking. Switch-based segmentation requires every device in the communication path to honour segment labels, creating a hard dependency on managed hardware. It is also constrained to a limited identifier space, insufficient for dense multi-tenant deployments. An overlay network removes these constraints by implementing logical segmentation in software. Traffic between virtual machines is handled by the virtualization layer, allowing segments to span multiple hosts without requiring changes to the underlying physical infrastructure. The physical network simply forwards standard traffic between hosts, while the logical topology is enforced above it.

This approach is particularly suitable in environments where the underlying network infrastructure cannot be modified or does not support advanced segmentation features. In scenarios where full control over physical switching equipment is available, similar isolation could be achieved using native mechanisms such as VLAN-based segmentation. However, the overlay model provides greater flexibility and portability, as it decouples the logical network design from the physical infrastructure.

Three-segment isolation model. The dual-tenant architecture requires network-level isolation, not merely application-level access control. A compromised component within one tenant’s boundary must be unable to reach another tenant’s resources without traversing a controlled routing point. Three logical segments are therefore defined: one for the first tenant’s workloads (e.g., VLAN 1), one for the second tenant’s workloads (e.g., VLAN 2), and one reserved exclusively for the observability and monitoring infrastructure. The dedicated monitoring segment prevents observability traffic from competing with application traffic, and keeps the monitoring stack reachable even when a segment is under stress — precisely when visibility is most critical. Figure 2.5 illustrates the resulting topology.

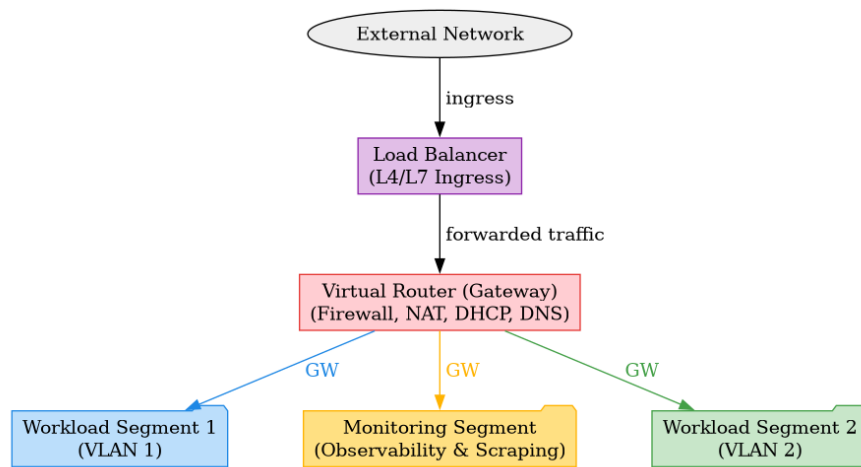


Figure 2.5: Virtual network topology. Ingress traffic enters through the load balancer and is forwarded to the virtual router, which enforces a default-deny firewall policy and acts as the gateway for the three isolated overlay segments. The monitoring segment is granted controlled read access to other segments and infrastructure components for observability purposes.

Routing and access control. The architecture introduces a central routing component that enforces segmentation and access control between logical networks. While traditional architectures often rely on physical appliances for inter-segment routing, this design utilizes a dedicated virtual router. This ensures tight integration with the software-defined overlay, providing fine-grained control over traffic flows without requiring overlay-aware physical hardware.

Ingress traffic from the external network first passes through a load balancer responsible for L4/L7 traffic distribution, which then forwards requests to the virtual router acting as the cluster’s gateway. By holding an interface in every internal segment as well as the external network, this gateway becomes the mandatory transit point for all ingress, egress, and inter-segment traffic. It enforces a *default-deny* posture: unmatched traffic is silently dropped. Cross-segment communication requires explicit bidirectional rules, ensuring isolation

by default. The monitoring segment may initiate targeted scrape connections into other segments and infrastructure components, but not the reverse, preventing a compromised component from disrupting observability. Outbound internet access is provided through address translation on the router’s upstream interface, keeping internal addresses hidden from external networks. A centralised address assignment service and integrated name resolution ensure consistent connectivity across all segments, decoupling service identity from instance placement.

External access and management integration. Administrative and management systems reside outside the logical segments and have no inherent route to addresses within the overlay. Virtual machines use the virtual router as their default gateway, so reply traffic flows correctly without additional configuration. Requests originating from management systems must be directed into the overlay explicitly. This is achieved by installing static routes on management hosts that direct traffic for overlay networks toward the router’s external interface, bypassing the load balancer which is reserved for application ingress. Alternatively, these routes can be configured on an upstream router, extending reachability to all devices on the management network without per-host configuration. Both approaches ensure full, bidirectional connectivity between management systems and virtual machines.

2.3.3 Containerisation Layer

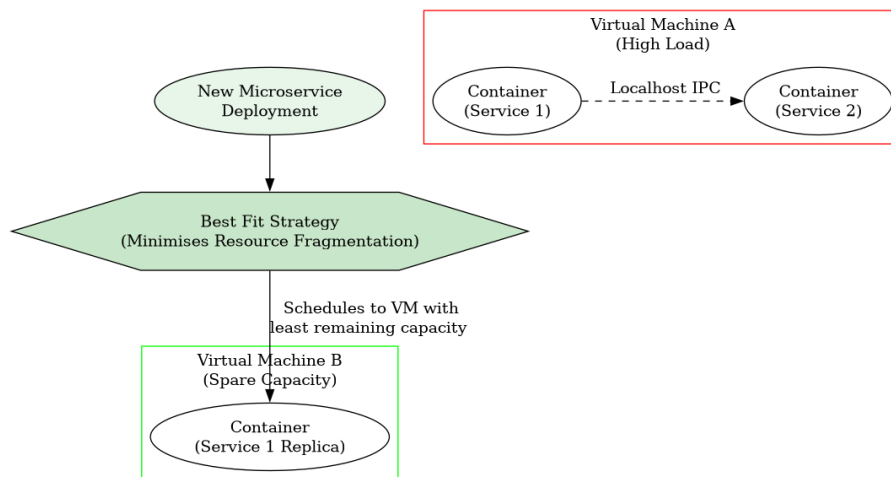


Figure 2.6: Layer 3: Containerisation, detailing the Best-Fit Strategy allocating microservices to minimize host fragmentation.

The containerisation layer deploys application services as containers distributed across the virtual machine fleet. Each container runs a single microservice and is isolated from co-located containers by the standard kernel mechanisms of the host operating system. Containers are mapped to virtual machines using a **Best Fit** strategy: for each container to be scheduled, the virtual machine that has the least remaining capacity while still being

able to satisfy the container's resource request is selected. This strategy minimises resource fragmentation across the VM fleet by filling existing virtual machines as fully as possible [10] before utilising the spare capacity of less-loaded ones, thereby reducing the number of virtual machines that must remain active to host a given set of containers and lowering the total energy and licensing cost of the cluster [?].

Containers belonging to microservices within the same mini-cluster are preferentially scheduled on virtual machines within that mini-cluster, preserving the network locality properties that the partitioning strategy was designed to create. When a horizontal scaling event creates a new virtual machine, its containers are placed according to the same Best Fit rule, and the new VM is registered within the same mini-cluster as its parent so that the cluster topology remains consistent.

2.3.4 Monitoring Layer

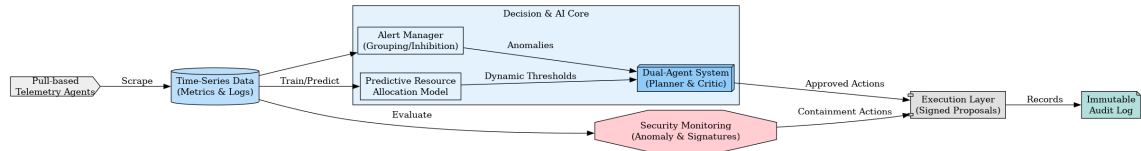


Figure 2.7: Layer 4: Monitoring and Security, outlining the telemetry ingestion, AI decision routing, and execution constraints.

The monitoring layer observes the operational state of all three layers beneath it and produces the information and automated responses required to maintain the cluster in its desired condition. It is composed of two functional modules: the monitoring subsystem and the alert manager.

Monitoring Subsystem

The monitoring subsystem collects telemetry data continuously from every observable entity in the cluster. Its collection scope spans all four categories of observable infrastructure: physical servers, including CPU utilisation, memory consumption, disk I/O throughput, and energy draw per node, aligning with modern Data Center Infrastructure Monitoring (DCIM) requirements [7]; virtual machines, including per-VM CPU steal time, working-set size, and container runtime metrics; physical network devices, including switch port utilisation, packet loss counters, and routing table state; and virtual network devices, including software-defined switch throughput and inter-VM traffic volumes. This unified collection scope ensures that no layer of the infrastructure produces blind spots in the operator's view of cluster state.

Data is collected through a pull-based model in which a central aggregator queries agents deployed on each node at configurable intervals, following the principle established

by network-adaptive proxy architectures that scale more predictably than push-based ones at large fleet sizes [?]. Collected streams are stored with time-windowed retention policies that preserve high-resolution data for anomalous intervals and apply progressive downsampling to normal intervals, consistent with the selective compression approaches seen in robust distributed commodity clusters [?]. Co-located streams — for example, the CPU and memory utilisation of all virtual machines sharing the same physical host — are stored with their correlational relationships preserved, enabling anomaly detection based on breaks in the correlation structure rather than on isolated threshold crossings alone.

Alert Manager

The alert manager is the decision module of the monitoring layer. It receives signals from the monitoring subsystem and is responsible for mapping observed conditions to specific remediation actions. When a monitored metric crosses a defined threshold or an anomaly model identifies a statistically abnormal pattern, the monitoring subsystem forwards a structured event to the alert manager, describing the affected resource, the observed value, and the severity classification.

The alert manager evaluates each incoming event against its rule base and selects the appropriate response. For scaling events — whether horizontal scaling triggered by elevated error rates and packet loss, or vertical scaling triggered by CPU, RAM, or I/O saturation — it issues the corresponding provisioning or resource-adjustment request to the virtual layer, as described in the scaling subsection above. For informational conditions that do not require automated remediation, it routes a notification to the operator interface, providing the contextual data needed for human review. To prevent alert storms during cascading failures, the alert manager applies grouping and inhibition rules that consolidate related events into a single actionable signal, suppressing downstream symptom alerts when a root-cause alert for the same component is already active [?].

2.4 Artificial Intelligence and Security

The management of a large-scale cluster involves operational complexities that exceed the capacity of static rules and manual human oversight. To address this, the architecture introduces a two-tiered Artificial Intelligence (AI) layer: a predictive mathematical component for continuous resource optimisation, and an autonomous cognitive agent system for decision-making and automated remediation.

2.4.1 Dynamic Thresholding and Predictive Resource Allocation

Traditional cluster monitoring relies on static thresholds (e.g., triggering an alert when CPU exceeds 80%). However, static rules fail to account for the shifting baseline of modern

microservices under varying load conditions. To resolve this, the cluster incorporates a predictive learning component that continuously adapts operational parameters.

Rather than waiting for resources to saturate, the system employs continuous regression modelling to predict optimal configurations based on high-frequency telemetry. This predictive component serves two functions:

- **Dynamic Alert Thresholds:** It predicts optimal warning and critical thresholds for computational resources (CPU, Memory) and application metrics (error rates). Warning thresholds are dynamically sized to trigger early soft-provisioning, while critical thresholds are tightened or loosened based on the cluster’s current stability.
- **Adaptive Fitness Weights:** The multi-objective provisioning algorithm relies on fitness weights (balancing CPU, RAM, I/O, and energy). The predictive model continuously calculates the current structural bottleneck of the cluster and dynamically adjusts these weights. If the cluster is globally constrained by memory, the algorithm automatically prioritises RAM preservation in subsequent placement decisions.

To prevent catastrophic forgetting and adapt to the evolving nature of the application, the models employ incremental learning. At regular intervals, the models ingest the latest batch of cluster metrics and update their internal representations using their own recent predictions as soft labels. This creates a self-supervised loop that adapts to new baseline behaviours without requiring external ground-truth datasets.

2.4.2 Autonomous Decision-Making and Risk-Based Escalation

While the predictive models configure the cluster’s parameters, automated remediation requires contextual reasoning. This is handled by a dual-agent cognitive system that acts as an autonomous operator. The system is designed around a strict segregation of duties between a *Planner* and a *Critic*.

The Planner continuously observes the cluster’s state, ingesting telemetry, predictive bottlenecks, and historical logs. When it detects anomalies or resource pressure, it reasons over the data to formulate a remediation strategy. It proposes a specific action (e.g., horizontally scaling a service or vertically resizing a virtual machine) and assigns it an initial risk classification.

The Critic acts as an independent safety validator. It receives the Planner’s proposed action and evaluates it against structural constraints. It verifies node headroom to ensure the action will not overload a physical server, checks for duplicate actions in recent logs to prevent runaway execution loops, and ensures the intervention matches the predicted bottleneck. The Critic either approves the action, rejects it with a suggested alternative, or escalates its risk level.

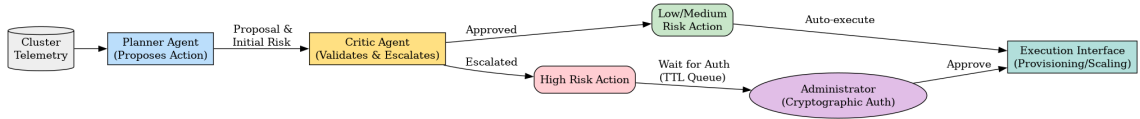


Figure 2.8: Cognitive dual-agent system highlighting the strict segregation of duties and the risk-based escalation matrix for automated decision-making.

To balance autonomy with safety, the architecture enforces a strict **Risk-Based Escalation Matrix**:

- **Low and Medium Risk:** Actions that are additive, reversible, or standard (e.g., adding resources, cloning instances for rapid pre-provisioning [12]) are executed immediately by the system once approved by the Critic. The human administrator is bypassed but informed via automated logs.
- **High Risk:** Actions that are destructive, irreversible, or highly disruptive (e.g., deleting virtual machines, catastrophic scale-downs) trigger a Human-in-the-Loop protocol. The action is placed in an approval queue, and the administrator receives a detailed alert explaining the reasoning, the expected effect, and the rollback strategy. The system waits for explicit human cryptographic approval before proceeding.

2.4.3 Security

Security in this cluster is not a peripheral concern appended after the functional design is complete; it is a structural requirement woven into every layer of the architecture. An adversary capable of manipulating telemetry data could silently disable automated remediation; one capable of issuing forged management commands could destroy production workloads; one capable of compromising the cognitive agents could turn the autonomous decision system into a tool of sabotage. The security design therefore addresses four orthogonal threat surfaces simultaneously: the network perimeter and internal traffic flows, the integrity and confidentiality of all communications, the patch posture and operational hygiene of every component, and the cognitive agent layer itself.

The unifying principle is *defence in depth* [11]: no single protective mechanism is trusted to be sufficient, and the failure of any one control does not expose the cluster to unacceptable risk. Where a control cannot prevent a threat entirely, it is designed to detect it and limit its consequences.

Network Security Architecture

The cluster’s network is partitioned into three logically isolated segments corresponding to the three operational domains it serves: the first tenant’s workloads, the second tenant’s workloads, and the shared monitoring infrastructure. This segmentation is not merely

organisational; it is enforced at the packet-routing level so that a compromise of one segment cannot propagate laterally to another without crossing an explicit control boundary.

At the external perimeter, a stateful traffic filter inspects all ingress and egress flows. Inbound traffic is subject to an allowlist: only the ports and protocols required by explicitly provisioned services may enter the cluster. Egress filtering applies an equivalent restriction, preventing a compromised internal component from establishing unauthorised outbound channels, which would otherwise be the first step of a data exfiltration or command-and-control scenario. This bidirectional filtering enforces the principle of least privilege at the network level.

Within the cluster, microsegmentation further constrains lateral movement. Each network segment imposes independent forwarding rules: traffic originating in the monitoring segment may observe the other segments through read-only metric collection but may not issue control commands to them. Traffic between the two tenant segments is prohibited by default; any inter-tenant flow would require an explicit policy exception that must be reviewed and justified. This containment strategy limits the blast radius of any single compromised container or virtual machine: an attacker gaining control of a workload in one segment cannot freely pivot to the others [11].

A local name-resolution service replaces reliance on external public resolvers. All internal hostnames are resolved within the cluster boundary, eliminating the risk of DNS hijacking through a compromised or malicious public resolver. Because service-to-service communication relies on resolved names, this control also prevents an attacker who has compromised a public DNS record from redirecting internal traffic to an adversary-controlled host.

Availability is protected by an anti-distributed-denial-of-service layer positioned at the perimeter before traffic reaches the internal components. Volumetric and protocol-level attacks are absorbed and filtered at this boundary, ensuring that a sustained flooding attack against a publicly reachable entry point does not saturate the cluster's internal networking capacity and thereby prevent legitimate service delivery.

Communication Integrity and Confidentiality

Every communication channel between cluster components — between agents and the management orchestrator, between the monitoring subsystem and the data aggregator, between the orchestrator and the provisioning layer, and between the system and the human administrator — is protected by a *digital signature* scheme. Each message is accompanied by a cryptographic signature generated with the sender's private key. The recipient verifies the signature before processing the message, guaranteeing both authenticity (the message originated from the claimed sender) and integrity (the message was not modified in transit). A message that fails verification is discarded, and the event is recorded in the audit log.

This mechanism is particularly critical for the communication path between the cognitive agents and the action execution layer. Because the agents reason about the cluster state and produce structured remediation proposals, an adversary who could inject forged proposals into this channel could induce arbitrary infrastructure modifications. The signature requirement ensures that only genuine proposals produced by the authenticated agent processes can proceed to the execution stage.

Traffic confidentiality is enforced through encrypted channels for all inter-component communications. Plaintext channels are not permitted, even on internal network segments, because internal segment compromise is treated as a credible threat model. Encryption prevents an attacker who has gained passive observation of internal traffic from extracting credentials, metric data, or management payloads.

Patch Management and Operational Hygiene

A foundational assumption of the security design is that every software component in the cluster will periodically contain newly disclosed vulnerabilities. The patch management strategy therefore treats vulnerability exposure as a continuous variable to be minimised, not as a binary state. A scheduled automated process runs at a fixed daily interval and applies all available security patches to every layer of the stack: the hypervisor host environment, the container runtime, the orchestration control plane, the monitoring subsystem, and the auxiliary services. This daily cadence ensures that the window between a public vulnerability disclosure and the deployment of its corresponding patch remains bounded.

All software components are sourced exclusively from their official distribution channels. The use of unofficial builds, community re-packaged variants, or versions obtained from unverified third-party repositories is prohibited. This policy eliminates the supply-chain risk of deploying components that have been tampered with before reaching the cluster. Wherever a component offers a mechanism to verify the authenticity of its distribution artefact — such as a signed package manifest or a content-addressed image digest — that mechanism is validated before installation.

The combination of daily patching and verified sourcing is complemented by the runtime monitoring described below: even if a vulnerability is exploited in the interval between its disclosure and the application of its patch, the anomaly detection subsystem is designed to surface the resulting abnormal behaviour before the attacker can consolidate their position.

Runtime Anomaly Detection and Security Monitoring

The monitoring subsystem described in the preceding section serves a dual purpose: it provides the operational telemetry that drives the resource management layer, and it serves as the primary source of security signals for the cluster. A dedicated security monitoring

agent operates continuously, subscribing to the same telemetry streams as the alert manager but applying a different analytical lens oriented toward behavioural deviation rather than resource saturation.

The agent evaluates the state of every observable component — hypervisor nodes, virtual machines, container workloads, network devices, and software-defined infrastructure — against both static integrity baselines and dynamically learned behavioural profiles. Deviations from expected behaviour, such as unusual process execution patterns, unexpected network connections originating from workload containers, anomalous spikes in log emission rates, or modifications to critical configuration files, are classified by severity.

A **critical** classification triggers an immediate alert routed directly to the system administrator, bypassing any aggregation or batching mechanisms. The alert carries sufficient contextual information for the administrator to evaluate the scope of the incident and take immediate containment action. A **warning** classification appends the anomalous event to a curated warning queue. The warning queue is reviewed and dispatched to the administrator at a configured periodic interval, allowing human attention to be applied at a cadence appropriate to the lower urgency of these signals without overwhelming the administrator with real-time notifications for every minor deviation. This two-tier severity routing mirrors the distinction between immediate remediation and periodic human review that governs the resource management alert manager, ensuring a consistent operational model across both functional domains.

Cognitive Agent Security

The cognitive agent system introduces a qualitatively distinct security challenge: the agents are autonomous decision-makers with the authority to propose infrastructure modifications. The security design for this layer addresses three specific threat vectors: unauthorised use of the agent interface, privilege escalation through adversarial prompting or model manipulation, and the systemic risk of fully autonomous execution of high-consequence actions.

Administrator Authentication. Access to the cognitive agent interface is gated by explicit administrator authentication. Before any natural-language query can be submitted to the Planner, the requesting party must demonstrate possession of valid credentials. In deployments with a single administrator, this requirement functions as a standard access control. In deployments with multiple administrators, access is further governed by a role-based authorisation model: each administrator is assigned a set of permitted operations, and the agent interface enforces these permissions before forwarding any request to the agent pipeline. This ensures that the authority to trigger cluster management operations through the agent is scoped to the same level as the authority to perform those operations manually.

Privilege Escalation Prevention. The Critic agent enforces a monotonic risk classification constraint: it may escalate the risk level assigned by the Planner, but it is constitutionally prohibited from lowering it. This one-directional constraint is a structural defence against adversarial jailbreaking scenarios in which a malicious input causes the Planner to misclassify a high-consequence action as low-risk in order to bypass the Human-in-the-Loop approval gate. Even if the Planner’s classification is compromised, the Critic’s independent evaluation and its inability to reduce the risk level ensure that the escalation path remains intact. An action that the Planner mislabels as low-risk but that the Critic correctly identifies as high-risk will still be routed to the administrator approval queue.

Furthermore, the cognitive agents do not possess direct credentials to the infrastructure management interfaces. They produce structured remediation proposals that are submitted to a central orchestration layer. This orchestration layer independently validates the execution context, verifies that the proposed action falls within the permitted parameter space, and only then dispatches the corresponding command to the provisioning interface. The agents are therefore isolated from direct infrastructure access; a compromised or hallucinating agent can produce a proposal, but cannot bypass the validation layer to execute it.

Human-in-the-Loop as a Security Boundary. The Human-in-the-Loop escalation mechanism described in the autonomous decision-making subsection functions as both an operational safety mechanism and a security boundary. High-risk actions — those classified as destructive, irreversible, or highly disruptive — are placed in an approval queue and cannot proceed until the administrator provides explicit authorisation. This authorisation is not merely a notification acknowledgement; it is a deliberate decision gate that requires the administrator to review the full context of the proposed action: the Planner’s reasoning, the Critic’s evaluation, the expected effect, and the prescribed rollback strategy.

The approval queue entries are bounded by a time-to-live: a high-risk action that does not receive administrator authorisation within a defined window is automatically rejected and recorded as expired. This TTL mechanism prevents an adversary from poisoning the approval queue with a malicious high-risk action and then waiting indefinitely for an administrator to accidentally approve it during a moment of inattention. Expired actions are logged with their full provenance, providing a forensic record of attempted but unauthorised operations.

Immutable Audit Trail. Every event in the pipeline — Planner proposals, Critic verdicts, execution outcomes, administrator approvals and rejections, and TTL expirations — is appended to an append-only audit log. This log is the authoritative record of every decision the autonomous system has made and every action the human administrator has taken.

It serves three security functions: it enables post-incident forensic reconstruction of the sequence of events leading to any infrastructure state; it provides the evidence base for detecting patterns of misuse or gradual privilege escalation that would be invisible in any single event; and it creates accountability, ensuring that the provenance of every infrastructure change — whether autonomous or human-initiated — is permanently traceable [?, 11].

Virtualisation and Containerisation Restrictions

The virtualisation and containerisation layers are subjected to a set of hardening constraints that reduce the attack surface exposed by the isolation mechanisms they provide. Virtual machines are configured with the minimum set of virtual hardware devices required for their workloads; unused device types are removed to eliminate associated driver attack surfaces. Hypervisor-level resource isolation is configured to prevent any single tenant's workloads from monopolising physical resources and thereby inducing a denial-of-service condition on co-located tenants.

At the container layer, each workload is granted only the kernel capabilities strictly necessary for its function. The default set of capabilities available to a container process is restricted to prevent privilege escalation exploits that rely on overly permissive capability sets. Container filesystems are mounted read-only wherever the workload does not require write access, reducing the persistence of any artefacts an attacker might attempt to install. Inter-container communication is governed by explicit network policy rules that restrict which workloads may communicate with which, consistent with the microsegmentation architecture described above. Furthermore, secure placement strategies are critical to minimize co-residency threats and side-channel attacks among heterogeneous workloads [11].

The monitoring and load-balancing infrastructure components, which require elevated access to observe the behaviour of other workloads, are isolated in their own network segment and run under dedicated service identities whose scope of authority is limited to observation and metric collection. They do not possess credentials to issue provisioning commands, ensuring that a compromise of the monitoring layer cannot be leveraged to modify the cluster topology.

2.5 Synthesis

The three pillars of this conception are mutually dependent rather than independent. The proof of concept validates the provisioning, scaling, and communication mechanisms that the monitoring and service placement layer depends upon for correct operation. The monitoring layer, in turn, provides the real-time utilisation data that the fitness function

requires for accurate placement decisions, the anomaly signals that drive the alert manager, and the historical telemetry on which the predictive AI models are trained. The security layer protects the integrity of the monitoring data itself: an adversary capable of manipulating the metrics seen by the alert manager or the predictive model could disable both automated remediation and intelligent assistance, making security not a peripheral concern but a structural requirement of the entire design. Together, the four architectural layers — physical, virtual, containerisation, and monitoring — and the three operational pillars form a coherent, self-regulating infrastructure capable of sustaining a high-load web application at scale while respecting the twin constraints of cost efficiency and high availability.

Bibliography

- [1] G. Balabanov and N. S. Apostolov, “Data Centre Infrastructure Monitoring,” in *60th International Scientific Conference on Information, Communication and Energy Systems and Technologies (ICEST)*, IEEE, 2025.
- [2] S. Mirjalili and A. Lewis, “The Whale Optimisation Algorithm,” *Advances in Engineering Software*, vol. 95, pp. 51–67, 2016.
- [3] R. Storn and K. Price, “Differential Evolution — A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces,” *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997.
- [4] X. Lai, X. Zheng, and W. Lu, “Energy-aware Virtual Machine Migration based on Dual Thresholds of Server Resource Utilization Rate considering Performance Interference,” in *2025 IEEE 2nd International Conference on Big Data Science and Engineering (ICBDSE)*, IEEE, 2025.
- [5] P. Zhang, M. Zhou, and X. Wang, “An Intelligent Optimization Method for Optimal Virtual Machine Allocation in Cloud Data Centers,” *IEEE Transactions on Automation Science and Engineering*, vol. 17, no. 4, pp. 1725–1735, 2020.
- [6] R. Begam, W. Wang, and D. Zhu, “TIMER-Cloud: Time-Sensitive VM Provisioning in Resource-Constrained Clouds,” *IEEE Transactions on Cloud Computing*, vol. 8, no. 1, pp. 297–311, 2020.
- [7] D. Saxena, I. Gupta, J. Kumar, A. K. Singh, and X. Wen, “A Secure and Multiobjective Virtual Machine Placement Framework for Cloud Data Center,” *IEEE Systems Journal*, vol. 16, no. 2, pp. 3163–3174, 2022.
- [8] J. Sung, S. Han, and J. Kim, “Cloning-based virtual machine pre-provisioning for resource-constrained edge cloud server,” *Cluster Computing*, vol. 27, pp. 1831–1847, 2024.
- [9] K. Senjab, S. Abbas, N. Ahmed, and A. R. Khan, “A survey of Kubernetes scheduling algorithms,” *Journal of Cloud Computing*, vol. 12, no. 87, 2023.

- [10] A. Bompotas, N.-R. Kalogeropoulos, and C. Makris, "CommC: A Multi-Purpose COMModity Hardware Cluster," *Future Internet*, vol. 17, no. 121, 2025.
- [11] H. Gu, J. Wang, J. Yu, D. Wang, B. Li, X. He, and X. Yin, "Towards virtual machine scheduling research based on multi-decision AHP method in the cloud computing platform," *PeerJ Computer Science*, vol. 9, e1675, 2023.
- [12] S. Tuli, G. Casale, and N. R. Jennings, "CILP: Co-simulation based Imitation Learner for Dynamic Resource Provisioning in Cloud Computing Environments," *arXiv preprint*, arXiv:2302.05630v2, 2023.
- [13] Q.-M. Nguyen, L.-A. Phan, and T. Kim, "Load-Balancing of Kubernetes-Based Edge Computing Infrastructure Using Resource Adaptive Proxy," *Sensors*, vol. 22, no. 2869, 2022.
- [14] R. Yang, Z. Li, J. Qian, and Z. Li, "Task-Driven Virtual Machine Optimization Placement Model and Algorithm," *Future Internet*, vol. 17, no. 73, 2025.
- [15] Y. Alahmad and A. Agarwal, "Multiple objectives dynamic VM placement for application service availability in cloud networks," *Journal of Cloud Computing*, vol. 13, no. 46, 2024.
- [16] D. Saxena and A. K. Singh, "A proactive autoscaling and energy-efficient VM allocation framework using online multi-resource neural network for cloud data center," *Neurocomputing*, 2022.